OH YEAH! — Documentazione Completa v4.0

Sistema di Analisi Fatture e Controllo Costi per la Ristorazione

Versione: 4.0
Ultimo aggiornamento: Marzo 2026
Autore: Mattia D'Avolio
URL Produzione: https://ohyeah.streamlit.app/

Indice

- Panoramica del Progetto
- Business Model e Proposta di Valore
- Architettura del Sistema
- Stack Tecnologico
- Struttura del Codice Sorgente
- Funzionalità Complete dell'Applicazione
- Pipeline di Classificazione AI
- Sistema di Autenticazione e Sicurezza
- Parsing delle Fatture Elettroniche
- Multi-Tenancy e Multi-Ristorante
- Schema Database Completo
- Pannello di Amministrazione
- Calcolo Marginalità e KPI
- Pagine Secondarie
- Testing e Qualità del Codice
- Deploy e Infrastruttura
- Monitoraggio e Alerting
- Sicurezza e Compliance GDPR
- Troubleshooting e FAQ Tecniche

---

1. Panoramica del Progetto

Cos'è OH YEAH!

OH YEAH! è una piattaforma SaaS web-based progettata specificamente per ristoratori italiani che necessitano di analizzare, categorizzare e controllare i costi derivanti dalle fatture elettroniche dei propri fornitori.

L'applicazione consente di:

- Caricare fatture elettroniche nei formati XML (FatturaPA), P7M (firma digitale CAdES), PDF e immagini (JPG/PNG)
- Classificare automaticamente ogni riga di fattura in una delle 29 categorie merceologiche tramite AI + regole deterministiche
- Visualizzare dashboard interattive con KPI, grafici a torta, pivot mensili e confronti tra fornitori
- Calcolare il Margine Operativo Lordo (MOL) con tabelle ricavi/costi mensili e analisi centri di produzione
- Gestire più ristoranti sotto un singolo account utente (multi-ristorante)
- Controllare i prezzi dei prodotti confrontando diversi fornitori
- Esportare dati in formato Excel per analisi esterne

Pubblico Target

Segmento	Descrizione
Ristoratori	Proprietari di ristoranti, pizzerie, bar, pasticcerie
Catene piccole	Aziende con 2-5 locali sotto la stessa P.IVA o P.IVA diverse
Consulenti F&B	Professionisti che seguono più locali

Table 1: Target di mercato OH YEAH!

Problema Risolto

I ristoratori italiani ricevono decine o centinaia di fatture elettroniche XML al mese dai fornitori. Queste fatture contengono righe di prodotti con descrizioni spesso abbreviate, non standardizzate e difficili da classificare. **OH YEAH! automatizza completamente l'analisi di queste fatture**, trasformando dati grezzi XML in informazioni azionabili per il controllo dei costi.

---

2. Business Model e Proposta di Valore

Modello di Revenue

Il servizio è attualmente in fase di lancio. Il modello previsto è **SaaS a subscription mensile** per cliente/ristorante.

Value Proposition

1. **Automazione totale**: Da file XML grezzi a dashboard in un click
2. **AI specializzata**: Classificazione addestrata su 600+ keyword del settore ristorazione italiano
3. **Apprendimento continuo**: Il sistema impara dalle correzioni manuali (admin e utenti)
4. **Multi-ristorante**: Un account, molti locali
5. **Zero installazione**: Web-based, accessibile da qualsiasi browser
6. **Conformità italiana**: Supporta FatturaPA, P7M firmati, validazione P.IVA italiana

Metriche Chiave

Metrica	Valore
Categorie merceologiche	29 (25 F&B + 1 Materiali + 3 Spese)
Keyword nel dizionario	600+ regole deterministiche
Formati fattura supportati	XML, P7M, PDF, JPG, PNG
Modello AI	OpenAI GPT-4o-mini
Copertura test automatici	149 test, 8 module di test
Tempo medio classificazione	< 5 secondi per 50 prodotti (batch)

Table 2: Metriche tecniche della piattaforma

3. Architettura del Sistema

Diagramma di Flusso Generale

Il sistema segue un'architettura a 3 livelli con frontend e backend integrati su Streamlit Cloud:

```
┌──────────────
```

```
│ GPT-4o- │  │PostgreSQL│  │ SMTP │
│ mini │  │ + RLS │  │ API v3 │
└──────────────┘  └──────────────┘  └──────────────┘
```

Pattern Architetturali

Pattern	Dove	Descrizione
MVC-like	Globale	Pages = View, Services = Model/Controller, Utils = Helper
Singleton	get_supabase_client()	Unica istanza Supabase per sessione
Dependency Injection	Tutti i service	supabase_client=None come parametro opzionale
Cache-aside	@st.cache_data	TTL 120s per fatture, 300s per margini
3-tier Memory	ai_service.py	Admin > Locale > Globale per classificazione
Thread-safe Lock	_cache_lock	threading.Lock() per dati condivisi
Batch Processing	Classificazione AI	50 articoli per chiamata API
Rate Limiting	Login + Reset + Upload + AI	Limiti in-memory con TTL

Table 3: Pattern architetturali implementati

---

4. Stack Tecnologico

Runtime e Framework

Componente	Tecnologia	Versione	Note
Linguaggio	Python	3.12.8	Type hints, f-strings, walrus operator
Framework Web	Streamlit	latest	SPA con auto-reload
Database	Supabase (PostgreSQL 15)	Free tier	EU region, Row Level Security
AI/ML	OpenAI API	GPT-4o-mini	Batch classification
Email	Brevo SMTP API v3	Free tier	300 email/giorno
Hashing	Argon2id	m=65536, t=3, p=4	OWASP raccomandato 2026
CI/CD	GitHub Actions	—	Uptime check ogni 5 minuti
Deploy	Streamlit Community Cloud	Free tier	Auto-deploy da branch main

Table 4: Stack tecnologico completo

Dipendenze Python Principali

- **streamlit** - Framework web UI
- **supabase** - Client PostgreSQL managed
- **openai** - Client API GPT-4o-mini
- **argon2-cffi** - Password hashing sicuro
- **pandas** - Data processing e aggregazione
- **plotly** - Grafici interattivi dashboard
- **openpyxl** - Export Excel
- **xmltodict** - Parsing XML fatture
- **asn1crypto** - Estrazione XML da file P7M
- **PyMuPDF (fitz)** - Parsing PDF fatture
- **tenacity** - Retry logic per API OpenAI
- **extra-streamlit-components** - Cookie manager per sessioni

5. Struttura del Codice Sorgente

Organizzazione Filesystem

Il progetto segue una struttura modulare ben definita:

```
FCI_PROJECT/
│
├── app.py # Entry point principale (4.080 righe)
│
├── pages/ # Pagine multi-page Streamlit
│  ├── admin.py # Pannello admin (6 tab)
│  ├── 1_calcolo_margine.py # Calcolo MOL
│  ├── 2_workspace.py # Workspace multi-ristorante
│  ├── 3_controllo_prezzi.py # Confronto prezzi fornitori
│  ├── gestione_account.py # Cambio password
│  └── privacy_policy.py # Privacy Policy + ToS
│
├── services/ # Business logic layer
│  ├── __init__.py # get_supabase_client() singleton
│  ├── ai_service.py # Classificazione AI (~550 righe)
│  ├── auth_service.py # Login, password, reset (~450 righe)
│  ├── invoice_service.py # Parsing XML/P7M/PDF (~400 righe)
│  ├── db_service.py # Query Supabase + cache (~350 righe)
│  ├── margine_service.py # Calcoli MOL (~400 righe)
│  └── email_service.py # Brevo SMTP (~100 righe)
│
├── utils/ # Utility e helper functions
│  ├── formatters.py # Formattazione numeri, base64
│  ├── text_utils.py # Normalizzazione testo
│  ├── validation.py # Validazione diciture
│  ├── piva_validator.py # Validazione P.IVA italiana
│  └── ... # Altri helper
│
├── config/ # Configurazione centralizzata
│  ├── constants.py # 29 categorie, 600+ keyword
│  ├── logger_setup.py # Logging (50MB, 10 backup)
│  └── prompt_ai_potenziato.py # Prompt GPT
│
```

```
├── tests/ # Test automatici (pytest)
│   ├── conftest.py # Fixtures condivise
│   ├── test_ai_service.py # Test classificazione AI
│   └── ... # 8 moduli di test (149 test totali)
│
└── requirements-lock.txt # 91 pacchetti con versioni freezate
```

---

6. Funzionalità Complete dell'Applicazione

Dashboard Principale

La dashboard è il cuore dell'applicazione. Dopo l'autenticazione, l'utente accede a:

KPI Box (6 metriche)

- **Spesa Totale** - Somma di tutti gli importi nel periodo
- **Spesa F&B** - Solo categorie Food & Beverage
- **Fornitori F&B** - Numero fornitori distinti Food
- **Fornitori Sp.Gen.** - Numero fornitori Spese Generali
- **Spesa Generale** - Solo le 3 categorie operative
- **Media Mensile** - Spesa media per mese nel periodo

Navigazione a 3 Sezioni

1. Dettaglio Articoli

- Data editor Streamlit interattivo con tutte le righe fattura
- Colonna Categoria editabile con dropdown (29 opzioni)
- Sistema di raggruppamento prodotti unici (checkbox, default ON)
- Ricerca per Prodotto, Categoria o Fornitore
- Filtro Food & Beverage / Spese Generali / Tutti
- Colonna Fonte con icone: Memoria, AI, 🖐 Manuale
- Bottone "Salva Modifiche" con propagazione batch
- Export Excel con ordinamento selezionabile

2. Categorie

- Pivot mensile per categoria (tabella con mesi come colonne)
- Totali per categoria su tutto il periodo

- Filtro F&B / Spese / Tutti

3. 🏭 Fornitori

- Pivot mensile per fornitore
- Stesso layout e filtri della vista Categorie

Upload Fatture

- Drag-and-drop o click "Sfoglia"
- Formati: XML, P7M, PDF, JPG, JPEG, PNG
- Limite: 100 file per upload, 200 MB totale
- Validazione P.IVA destinatario vs ristorante attivo
- Progress bar in tempo reale durante l'elaborazione
- Rilevamento duplicati (stessa fattura già caricata)
- Log dettagliato per ogni file (successo/errore/scartato)

---

7. Pipeline di Classificazione AI

Architettura Memoria a 3 Livelli

Il sistema implementa una gerarchia di memoria intelligente per la classificazione:

1. **MEMORIA ADMIN** (classificazioni_manuali) - Priorità MASSIMA, scope globale per tutti i clienti
2. **MEMORIA LOCALE** (prodotti_utente) - Priorità ALTA, scope specifico per cliente
3. **MEMORIA GLOBALE** (prodotti_master) - Priorità MEDIA, scope tutti i clienti
4. **DIZIONARIO KEYWORD** (config/constants.py) - 600+ regole deterministiche
5. **AI GPT-4o-mini** - Ultima risorsa, batch da 50 articoli

Cache In-Memory Thread-Safe

Il sistema mantiene una cache in memoria per evitare il problema N+1 query:

```
_memoria_cache = {
'prodotti_utente': {},
```

```
'prodotti_master': {},
'classificazioni_manuali': {},
'version': 0,
'loaded': False
}
```

Caratteristiche:

- Caricamento lazy (1 volta per sessione, 3 query DB totali)
- Invalidazione esplicita dopo ogni modifica
- Thread-safe con threading.Lock()
- Elimina completamente il problema N+1 query

29 Categorie Merceologiche

Food & Beverage (25 categorie)

#	Categoria	Esempi
1	ACQUA	Acqua naturale, frizzante
2	AMARI/LIQUORI	Limoncello, Baileys, Sambuca
3	BEVANDE	Coca Cola, Aranciata, Succhi
4	BIRRE	Lager, Weiss, Stout
5	CAFFE E THE	Espresso, Capsule, Tisane
6	CARNE	Pollo, Manzo, Vitello, Salsiccia
7	DISTILLATI	Vodka, Gin, Whisky, Grappa
8	FRUTTA	Mele, Arance, Fragole
9	GELATI	Gelato, Sorbetto, Semifreddo
10	LATTICINI	Parmigiano, Mozzarella, Burrata

Table 5: Categorie Food & Beverage (primi 10)

(Nota: Le altre 15 categorie F&B includono: Olio e Condimenti, Pasticceria, Pesce, Prodotti da Forno, Salse e Creme, Salumi, Scatolame, Secco, Shop, Spezie, Sushi Varie, Uova, Varie Bar, Verdure, Vini)

Materiali (1 categoria)

26. **MATERIALE DI CONSUMO** - Tovaglioli, Pellicola, Guanti, Detersivo, Bicchieri

Spese Operative (3 categorie - NON Food & Beverage)

27. **SERVIZI E CONSULENZE** - Commercialista, HACCP, POS, Marketing
28. **UTENZE E LOCALI** - Bollette, Affitto, Telefono, Gas
29. **MANUTENZIONE E ATTREZZATURE** - Riparazione forno, Lavastoviglie, Arredi

Centri di Produzione

Il sistema aggrega le 29 categorie in 6 macro centri di produzione:

Centro	Categorie Incluse
FOOD	Carne, Pesce, Latticini, Salumi, Uova, Scatolame, Olio, ...
BAR	Acqua, Bevande, Caffè e The, Varie Bar
ALCOLICI	Birre, Vini, Distillati, Amari/Liquori
DOLCI	Pasticceria, Gelati
MATERIALE	Materiale di Consumo
SHOP	Shop

Table 6: Aggregazione macro per centri di produzione

---

8. Sistema di Autenticazione e Sicurezza

Flusso di Autenticazione

```
UTENTE → Cookie session_token (30 giorni)
│
▼
Cookie valido? ──YES──→ Login automatico
│
NO
▼
Form Login (email + password)
```

│
▼
Rate Limit check (5 tentativi → 15 min lockout)
│
▼
Verifica Argon2id hash
│
▼
Login OK → session_token generato + cookie 30 gg

Password Hashing

Il sistema utilizza **Argon2id**, algoritmo raccomandato da OWASP 2026:

Parametro	Valore
Algoritmo	Argon2id
Memory cost	65536 KB (64 MB)
Time cost	3 iterazioni
Parallelism	4 thread
Migrazione	Auto da SHA256 legacy al primo login

Table 7: Parametri password hashing

Validazione Password (GDPR Art.32)

La validazione implementa requisiti stringenti conformi alle linee guida del Garante Privacy italiano:

- Lunghezza minima: 10 caratteri
- Complessità: almeno 3/4 tra maiuscola, minuscola, numero, simbolo
- Blacklist: 24 password comuni (OWASP + varianti italiane come "ristorante", "pizzeria")
- No dati personali: email e nome ristorante vietati nella password
- No pattern semplici: sequenze numeriche e caratteri ripetuti bloccati

Rate Limiting

Funzione	Limite	Lockout
Login	5 tentativi	15 minuti
Reset password	1 richiesta	5 minuti cooldown
Upload file	100 file / 200 MB	Per singolo upload
AI classificazione	1.000 chiamate/giorno	Per ristorante

Table 8: Limiti di rate per funzionalità

Reset Password via Email

Il processo di reset password segue un flusso sicuro:

1. Utente inserisce email → sistema genera codice 6 cifre con secrets.randbelow()
2. Email inviata via Brevo SMTP API con codice e scadenza 15 minuti
3. Utente inserisce codice + nuova password → verifica HMAC constant-time
4. Password validata secondo compliance GDPR → hash Argon2id salvato atomicamente
5. Token reset invalidato → login automatico

---

9. Parsing delle Fatture Elettroniche

Formati Supportati

Formato	Metodo	Libreria	Note
XML (FatturaPA)	Parsing diretto	xmltodict	Encoding auto-detect
P7M (firma digitale)	Estrazione ASN.1	asn1crypto	CAdES/PKCS#7, max 50 MB
PDF	Estrazione testo + OCR	PyMuPDF + Vision	Fallback Vision
JPG/PNG	OCR Vision API	OpenAI Vision	Conversione base64

Table 9: Formati fattura supportati

Pipeline di Parsing XML

Il processo di parsing XML implementa 10 step sequenziali:

1. **Lettura file** - Byte stream dal file uploader Streamlit
2. **Rilevamento encoding** - Prolog XML → charset-normalizer → fallback UTF-8
3. **Parsing XML** - xmltodict.parse() → dizionario Python
4. **Estrazione metadati** - Data documento, tipo documento (TD01/TD04), fornitore
5. **Validazione P.IVA** - Confronto cedente/prestatore vs P.IVA ristorante attivo
6. **Estrazione righe** - Per ogni DettaglioLinee: descrizione, quantità, prezzo, IVA
7. **Filtro diciture** - Esclusione automatica righe informative (regex pattern)
8. **Calcolo prezzo effettivo** - totale_riga / quantità per gestire sconti
9. **Categorizzazione immediata** - Check memoria cache → keyword → "Da Classificare"
10. **Salvataggio batch** - Upsert su tabella fatture con dedup

Tipi Documento Gestiti

Codice	Tipo	Trattamento
TD01	Fattura	Importi positivi normali
TD02	Acconto su fattura	Importi positivi
TD04	Nota di Credito	Importi invertiti (negativi)
TD05	Nota di Debito	Importi positivi
TD06	Parcella	Importi positivi
TD16-TD27	Autofatture/Integrazioni	Importi positivi

Table 10: Gestione tipi documento FatturaPA

---

10. Multi-Tenancy e Multi-Ristorante

Isolamento Dati

Ogni query al database include obbligatoriamente il filtro `user_id` per garantire completo isolamento dei dati tra clienti:

```
supabase.table("fatture")
.select("*")
.eq("user_id", user_id) # Isolamento utente
.eq("ristorante_id", ristorante_id) # Isolamento ristorante
```

Inoltre, Supabase PostgreSQL ha **Row Level Security (RLS)** attivo su tutte le tabelle, che funge da secondo livello di protezione.

Architettura Multi-Ristorante

Un utente può avere più ristoranti (es. catena con 3 locali):

```
UTENTE (users)
└── Ristorante 1 (ristoranti) ──→ Fatture ristorante 1
└── Ristorante 2 (ristoranti) ──→ Fatture ristorante 2
└── Ristorante 3 (ristoranti) ──→ Fatture ristorante 3
```

Funzionalità:

- **Dropdown selezione** - Visibile solo con 2+ ristoranti
- **Persistenza ultimo usato** - Salvato in users.ultimo_ristorante_id
- **Pulizia contesto** - Al cambio ristorante, cache e stato sessione resettati
- **Admin** - Vede TUTTI i ristoranti di tutti i clienti

---

11. Schema Database Completo

Tabelle Principali

users - Utenti del sistema

Colonna	Tipo	Note
id	UUID (PK)	Generato automaticamente
email	TEXT UNIQUE	Login, lowercase
password_hash	TEXT	Argon2id hash
nome_ristorante	TEXT	Nome locale (legacy)
partita_iva	TEXT	P.IVA (legacy)
attivo	BOOLEAN	Account attivo/disattivato
session_token	TEXT	Token sessione cookie
ultimo_ristorante_id	UUID (FK)	Ultimo ristorante usato
pagine_abilitate	JSONB	"marginalita": true, ...

Table 11: Struttura tabella users

ristoranti - Locali (multi-ristorante)

Colonna	Tipo	Note
id	UUID (PK)	Generato automaticamente
user_id	UUID (FK → users)	Proprietario
nome_ristorante	TEXT	Nome locale
partita_iva	TEXT	P.IVA per validazione fatture
ragione_sociale	TEXT	Ragione sociale
attivo	BOOLEAN	Ristorante attivo

Table 12: Struttura tabella ristoranti

fatture - Righe di fattura (core data)

Colonna	Tipo	Note
id	BIGINT (PK)	Auto-increment
user_id	UUID (FK → users)	Proprietario
ristorante_id	UUID (FK → ristoranti)	Ristorante associato
file_origine	TEXT	Nome file originale
data_documento	DATE	Data documento fattura
fornitore	TEXT	Nome fornitore
descrizione	TEXT	Descrizione prodotto normalizzata
quantita	NUMERIC	Quantità
prezzo_unitario	NUMERIC	Prezzo per unità
totale_riga	NUMERIC	Importo totale riga
categoria	TEXT	Categoria assegnata
tipo_documento	TEXT	TD01, TD04, etc.

Table 13: Struttura tabella fatture

prodotti_master - Memoria globale AI

Colonna	Tipo	Note
id	BIGINT (PK)	Auto-increment
descrizione	TEXT UNIQUE	Chiave matching
categoria	TEXT	Categoria assegnata
classificato_da	TEXT	"AI", "keyword", "Utente (email)"
confidence	TEXT	"altissima", "alta", "media"
verified	BOOLEAN	Verificato da admin
volte_visto	INT	Counter occorrenze

Table 14: Struttura tabella prodotti_master

margini_mensili - Dati MOL

Colonna	Tipo	Note
id	BIGINT (PK)	Auto-increment
user_id	UUID (FK)	Proprietario
ristorante_id	UUID (FK)	Ristorante
anno	INT	Anno
mese	INT	Mese (1-12)
fatturato	NUMERIC	Fatturato totale lordo
costi_fb	NUMERIC	Costi Food & Beverage
costi_spese	NUMERIC	Costi Spese Generali
costi_personale	NUMERIC	Costo personale

Table 15: Struttura tabella margini_mensili

Altre Tabelle

Il sistema implementa ulteriori tabelle per funzionalità avanzate:

- **prodotti_utente** - Memoria locale cliente (personalizzazioni)
- **classificazioni_manuali** - Memoria admin (priorità massima)
- **upload_events** - Log upload fatture con tracking errori
- **ricette** - Ricette con ingredienti e costi
- **ingredienti_utente** - Ingredienti personalizzati
- **note_diario** - Diario operativo per ristorante
- **review_confirmed** - Righe confermate dopo review admin
- **review_ignored** - Righe ignorate in review admin

---

12. Pannello di Amministrazione

Il pannello admin è accessibile solo agli utenti con email in `ADMIN_EMAILS` e si articola in 6 tab principali:

Tab 1: □ Gestione Clienti

- Lista completa clienti con statistiche (fatture, righe, costi, ultimo caricamento)
- Creazione nuovo cliente GDPR-compliant (l'admin non imposta la password)

- Token di attivazione via email (24 ore validità)
- Impersonazione clienti (l'admin vede l'app come il cliente)
- Cookie impersonazione per sopravvivere a refresh pagina
- Attivazione/Disattivazione account
- Gestione pagine abilitate per cliente (Marginalità, Workspace)

Tab 2: □ Review Righe €0

- Lista righe fattura con prezzo = €0 (potenziali diciture/note)
- Classificazione diretta: "È un prodotto" (assegna categoria) o "È una dicitura" (escludi)
- Salvataggio in classificazioni_manuali con flag is_dicitura
- Review permanente: righe confermate e ignorate tracciate

Tab 3: □ Memoria Globale AI

- Data editor con tutti i record di prodotti_master
- Modifica categoria direttamente nella tabella
- Propagazione modifica a tutte le fatture nel database
- Filtro per descrizione e categoria
- Conteggio record e statistiche

Tab 4: □ Memoria Clienti

Visualizzazione `prodotti_utente` per ogni cliente, utile per diagnosticare classificazioni personalizzate.

Tab 5: □ Integrità Database

- Conteggio righe per tabella
- Verifica fatture con categoria NULL
- Verifica ristoranti senza utente
- Verifica utenti senza ristoranti
- Fix automatici disponibili

Tab 6: □ Costi AI

- Tracciamento costi OpenAI per cliente/ristorante
- Aggregazione giornaliera e mensile
- Budget giornaliero: 1.000 classificazioni per ristorante

---

13. Calcolo Marginalità e KPI

La pagina Calcolo Margine implementa analisi economica avanzata articolata in 3 sotto-tab:

Calcolo Ricavi-Costi-Margini

Tabella trasposta 12 mesi con voci automatiche e manuali:

Voce	Fonte	Formula
Fatturato Lordo	Input manuale	—
IVA Media %	Input manuale	—
Fatturato Netto IVA	Calcolato	Lordo × (1 - IVA%)
Costi F&B	Auto da fatture	Somma categorie Food
Costi Spese Generali	Auto da fatture	Somma 3 categorie spese
Costo Personale	Input manuale	—
Food Cost %	**Calcolato**	**(Costi F&B / Fatt. Netto) × 100**
1° Margine	**Calcolato**	**Fatturato Netto - Costi F&B**
MOL	**Calcolato**	**Fatt. Netto - F&B - Spese - Personale**
MOL %	**Calcolato**	**MOL / Fatt. Netto × 100**

Table 16: Struttura calcolo marginalità

Soglie KPI con Commenti Automatici

Il sistema valuta automaticamente le performance rispetto a soglie di settore:

KPI	Eccellente	Norma	Attenzione	Critico
Food Cost %	< 28%	28-33%	33-38%	> 38%
Spese Gen. %	< 15%	15-22%	22-28%	> 28%
1° Margine %	> 70%	62-70%	55-62%	< 55%
MOL %	> 20%	12-20%	5-12%	< 5%

Table 17: Soglie KPI settore ristorazione

Centri di Produzione

- Suddivisione fatturato per centro (FOOD, BAR, ALCOLICI, DOLCI)
- Calcolo Food Cost e margine per singolo centro
- Grafici a barre comparativi

Analisi Avanzate

- Breakdown costi per categoria nel periodo selezionato
- Pivot mese × categoria con totali e percentuali
- Export Excel formattato

---

14. Pagine Secondarie

Workspace Multi-Ristorante

- Overview di tutti i ristoranti dell'utente
- Note diario per ristorante (con sanitizzazione XSS via html.escape())
- Statistiche aggregate per locale

Confronto Prezzi

- Ricerca prodotto per descrizione
- Confronto prezzi tra fornitori diversi per lo stesso prodotto
- Storico prezzi nel tempo
- Alert variazioni significative

Gestione Account

- Cambio password con validazione GDPR
- Visualizzazione dati account
- Gestione ristoranti (nome, P.IVA, ragione sociale)

Privacy e Condizioni

Due tab distinti:

- **Privacy Policy** - Informativa GDPR completa (titolare, base giuridica, diritti, data retention)
- **Terms of Service** - Condizioni d'uso, limitazioni, clausola "non conservazione sostitutiva"

---

15. Testing e Qualità del Codice

Suite di Test (149 test totali)

Il sistema implementa una suite completa di test automatici con pytest:

File	Test	Copertura
test_ai_service.py	~20	Classificazione AI, memoria 3 livelli
test_auth_service.py	~25	Login, rate limiting, GDPR password
test_constants.py	~15	Integrità categorie, regex, KPI soglie
test_formatters.py	~15	Formattazione numeri, base64
test_invoice_service.py	~20	Parsing XML, P7M, encoding
test_piva_validator.py	~15	Validazione P.IVA (Luhn)
test_text_utils.py	~20	Normalizzazione, estrazione fornitore
test_validation.py	~19	Diciture, sconti, integrità fattura

Table 18: Distribuzione suite di test

Fixtures

Il file `conftest.py` implementa fixture condivise:

- **mock_supabase** - Mock completo del client Supabase con risposte predefinite
- **mock_openai** - Mock OpenAI che simula risposte di classificazione
- **sample_xml** - Fattura XML di esempio per test parsing
- Isolamento completo: nessun test tocca servizi esterni

Ultimo risultato esecuzione: **149/149 PASSED (0.62s)**

---

16. Deploy e Infrastruttura

Streamlit Community Cloud

Parametro	Valore
Piattaforma	Streamlit Community Cloud (Free Tier)
Repository	GitHub (branch main)
Auto-deploy	Push su main → deploy automatico
URL	https://ohyeah.streamlit.app/
Region	US (default Streamlit)
Python version	3.12

Table 19: Configurazione deploy

Supabase

Parametro	Valore
Piano	Free Tier
Region	EU (Frankfurt)
PostgreSQL	v15
RLS	Attivo su tutte le tabelle
Backup	Automatici giornalieri
Limite	500 MB storage, 2 GB transfer

Table 20: Configurazione Supabase

17. Monitoraggio e Alerting

GitHub Actions - Uptime Check

Il sistema implementa monitoring automatico ogni 5 minuti tramite GitHub Actions:

```
name: Uptime Check
on:
schedule:
- cron: '*/5 * * * *'
workflow_dispatch:
```

Il workflow effettua:

1. Chiamata HTTPS a https://ohyeah.streamlit.app/ con timeout 30s
2. Verifica HTTP status code (atteso: 200)
3. Se DOWN → Invio email alert via Brevo API
4. Log completo su GitHub Actions

Logging Applicativo

Componente	Configurazione
File handler	RotatingFileHandler
Max dimensione	50 MB per file
Backup files	10 (totale max ~550 MB)
Livello	INFO in produzione
Logger modulari	app, ai, auth, invoice, db, admin, email

Table 21: Configurazione logging

---

18. Sicurezza e Compliance GDPR

Misure di Sicurezza Implementate

Il sistema implementa un framework completo di sicurezza multi-livello:

Categoria	Misura	Dettaglio
Autenticazione	Argon2id	m=65536, t=3, p=4 (OWASP 2026)
Sessioni	Token UUID4 + Cookie 30gg	Invalidazione su logout
Rate Limiting	Login: 5/15min, Reset: 1/5min	In-memory thread-safe
Input Validation	Sanitizzazione AI	Control char removal + truncation
XSS Prevention	html.escape()	Su tutti output user-generated
CSRF Protection	enableXsrfProtection = true	Streamlit nativo
SQL Injection	Parametrized queries	Supabase client
File Upload	Magic bytes validation	Verifica header oltre estensione
Password	GDPR Art.32 compliance	10+ char, 3/4 complessità

Table 22: Framework di sicurezza

Compliance GDPR

Il sistema implementa tutti i requisiti normativi per la protezione dei dati:

- **Privacy Policy** - Pagina dedicata con informativa completa
- **Terms of Service** - Condizioni d'uso con clausole legali italiane
- **Data Retention** - Le fatture restano nel DB finché l'utente le elimina
- **Diritto all'oblio** - L'admin può eliminare completamente un account
- **Portabilità** - Export Excel di tutti i dati
- **Base giuridica** - Contratto (Art. 6.1.b) per il servizio
- **Nota legale** - "Non costituisce sistema di Conservazione Sostitutiva ai sensi del D.M. 17 giugno 2014"
- **Creazione client GDPR** - L'admin non conosce mai la password del cliente

19. Troubleshooting e FAQ Tecniche

Problemi Comuni

L'app non si carica (pagina bianca)

1. Verificare che Supabase non sia in pausa (free tier: 7 giorni inattività)
2. Controllare status su https://status.streamlit.io
3. Verificare GitHub Actions per alert automatici

Fattura scartata durante upload

- **"P.IVA non corrispondente"** - La P.IVA del cedente non corrisponde al ristorante attivo
- **"File già caricato"** - Deduplicazione basata su file_origine + user_id + ristorante_id
- **"Encoding non supportato"** - File con charset esotico → charset-normalizer rileva automaticamente

L'AI non classifica correttamente un prodotto

1. **Correggi manualmente** - Modifica la categoria nel data editor e clicca "Salva"
2. **Il sistema impara** - La correzione viene salvata in memoria locale (cliente) o globale (admin)
3. **Prossima volta** - Il prodotto sarà classificato correttamente senza chiamata AI

Limiti dell'Applicazione

Limite	Valore
Max file per upload	100
Max dimensione upload	200 MB
Max dimensione P7M	50 MB
Max righe per utente	100.000
Max chiamate AI/giorno	1.000 per ristorante
TTL cache fatture	120 secondi
TTL sessione cookie	30 giorni
Lockout login	15 minuti dopo 5 tentativi
Batch AI	50 articoli per chiamata

Table 23: Limiti tecnici del sistema

Conclusioni

OH YEAH! rappresenta una soluzione SaaS completa per il controllo costi nella ristorazione italiana, con:

- **Automazione totale** - Da XML a dashboard in pochi click
- **AI specializzata** - 600+ keyword + GPT-4o-mini addestrato sul settore
- **Memoria multi-livello** - Sistema che impara dalle correzioni
- **Conformità normativa** - GDPR compliant, validazione P.IVA, FatturaPA
- **Scalabilità** - Multi-ristorante, multi-tenant con RLS
- **Qualità enterprise** - 149 test automatici, logging avanzato, uptime monitoring

Per ulteriori informazioni: mattiadavolio90@gmail.com